

Qt 对战游戏与强化学习 AI

项目作业报告

第 22 组

组 号： 22

成员 A： 李子禾

成员 B： 吴狄隆

成员 C： 边扬博

日 期： 2026 年 6 月

目录

1	程序功能介绍	2
1.1	项目概述	2
1.2	核心玩法	2
1.3	游戏模式	2
1.4	其他功能特性	2
2	项目各模块与类设计细节	3
2.1	整体架构	3
2.2	游戏对象层	3
2.2.1	GameObject — 基类	3
2.2.2	Player — 玩家类	3
2.2.3	Bullet 与 Bomb	3
2.2.4	Platform	3
2.3	游戏管理层 — GameManager	4
2.4	游戏主循环 — GameWidget	4
2.5	AI 系统	4
2.5.1	网络结构 (SimpleMLP)	4
2.5.2	AIController	4
2.5.3	状态空间	4
2.6	强化学习训练系统 (Python 端)	5
2.6.1	算法: Double DQN	5
2.6.2	分阶段课程学习	5
2.6.3	奖励设计	5
2.6.4	环境与部署	5
2.7	音频与 UI 系统	5
3	小组成员分工	5
3.1	李子禾 — 玩家与攻击系统负责人	6
3.2	吴狄隆 — 界面与游戏画布负责人	6
3.3	边扬博 — 地图、碰撞、规则与 AI 负责人	6
3.4	分工汇总	7
4	项目总结与反思	7
4.1	项目成果	7
4.2	技术亮点	7
4.3	不足与反思	7
4.4	收获与体会	8

1 程序功能介绍

1.1 项目概述

本项目是一款基于 Qt 6 框架开发的**双人横版对战游戏**，集成了基于**深度强化学习 (Double DQN)** 训练的 AI 对手。项目包含两大子系统：

1. **Qt 游戏客户端 (C++)**：完整的游戏实现，包含物理引擎、碰撞检测、渲染系统、音频系统和 AI 推理模块。
2. **强化学习训练框架 (Python/PyTorch)**：DQN 训练管线，包含环境模拟、分阶段课程学习、自我对弈和权重导出。

1.2 核心玩法

游戏为**竞技场对战模式**：两名角色在由 9 个平台构成的多层地图上进行战斗，通过射击和投掷炸弹消耗对方生命，率先将对方全部生命耗尽的一方获胜。

- **移动与跳跃**：支持左右移动、二段跳（第二段跳高度为首段的 91%）、下键穿越平台。
- **射击系统**：弹夹容量 8 发，射击冷却 9 帧，装弹 78 帧；子弹造成 12 点伤害并附带击飞效果（水平 16.1、竖直 -10.5），射击有后坐力（2.9）。
- **炸弹系统**：抛物线投掷，105 帧（约 1.75 秒）后爆炸，爆炸半径 120 像素，造成 45 点伤害和方向性击飞（力度 18.0）。自身炸弹仅击飞不扣血，可用于”火箭跳”。
- **生命机制**：每人 100HP、5 条命。HP 归零或坠出屏幕（ $y > 705$ ）消耗一条命，复活后有 95 帧无敌时间。

1.3 游戏模式

模式	说明
人人对战	两名玩家分别使用键盘左右两侧按键进行对战
人机对战（简单）	玩家对战基于手写规则的 AI（子弹躲避、追踪射击等策略）
人机对战（困难）	玩家对战经过强化学习训练的神经网络 AI

1.4 其他功能特性

- **音频系统**：背景音乐循环播放，射击、爆炸等动作配有独立音效，支持独立音量调节与启用/禁用。
- **视觉效果**：着陆尘埃、枪口火焰、命中闪光、爆炸粒子、浮动伤害数字、子弹击中平台火花、屏幕震动、击杀提示等。
- **UI 系统**：渐变色主菜单、暂停菜单（含音量控制）、帮助说明界面、按键配置系统。
- **屏幕外指示**：当敌人离开可视区域时，屏幕边缘显示方向标记。
- **无敌闪烁**：复活和受击后角色闪烁提示无敌状态。

2 项目各模块与类设计细节

2.1 整体架构

项目采用**模块化分层设计**，各模块职责清晰。代码总量约 4000 行 C++ 与 2000 行 Python。

层次	核心文件	职责
游戏对象层	gameobject, player, bullet, bomb, platform	游戏实体的属性、物理行为与绘制
游戏管理层	gamemanager	碰撞检测、规则判定、生命管理
游戏主循环	gamewidget	输入处理、帧更新、渲染、AI 整合
AI 系统	ai_controller, simple_mlp, ai_types	神经网络推理与规则 AI
音频系统	audiomanager	音效与音乐的播放与管理
UI 系统	mainwindow, pausedialog, helpdialog	菜单界面与交互
配置系统	gameconfig	全局按键配置（单例模式）
RL 训练（Python）	train, dqn_model, duel_env 等	强化学习训练管线

2.2 游戏对象层

2.2.1 GameObject — 基类

所有游戏对象的基类，提供浮点坐标系统（`m_x`, `m_y`, `m_width`, `m_height`），支持 `QRect` 和 `QRectF` 双精度碰撞矩形，定义虚函数接口 `update()` 和 `draw()`。

2.2.2 Player — 玩家类

继承自 `GameObject`，是最复杂的游戏对象，包含三个子系统：

- **物理系统**：水平/竖直速度，地面加速度 0.62/空中 0.36，地面摩擦 0.78/空中 0.965，重力 0.54，最大下坠速度 12.6。支持二段跳和平台穿越。
- **武器系统**：弹夹管理（容量 8 发）、射击冷却（9 帧）、装弹机制（78 帧）、炸弹冷却（132 帧）。通过 `canShoot()`、`shootPosition()` 等接口供外部查询。
- **状态系统**：HP 管理、无敌计时器、死亡判定、朝向管理。`applyKnockback(vx,vy)` 实现击飞效果，`applyRecoil()` 实现射击后坐力。

2.2.3 Bullet 与 Bomb

`Bullet` 实现直线飞行的子弹（速度 16.5px/帧，生命 80 帧），碰撞后触发伤害和击飞。`Bomb` 实现抛物线运动的炸弹，具有重力物理（0.42）、倒计时（105 帧）、范围爆炸（半径 120px），并基于角度计算方向性击飞。自身炸弹仅击飞不扣血。

2.2.4 Platform

平台类，存储碰撞矩形，支持 `QRect`/`QRectF` 双格式。地图由 9 个平台组成对称的竞技场布局。

2.3 游戏管理层 — GameManager

负责碰撞检测与规则判定，核心功能：

- **平台碰撞** (`collidePlayerWithPlatforms`)：检测玩家是否落在平台上，支持下键穿越。对子弹、炸弹也有独立的平台碰撞处理。
- **命中检测** (`checkBulletHits/checkBombHits`)：子弹命中触发伤害和击飞（仅敌方子弹生效，无敌状态免疫）；炸弹范围伤害基于圆形碰撞和角度计算击飞方向。
- **生命管理** (`checkLifeState`)：检测坠亡和血量归零，管理复活、生命计数和游戏结束判定。

2.4 游戏主循环 — GameWidget

继承 `QWidget`，使用 `QTimer` 以 16ms 间隔（约 60FPS）驱动。每帧执行流程：

1. **输入处理**：P1 读取键盘状态 (`QSet<int>` 记录按下的键)；P2 根据模式选择人类输入或 AI 决策。
2. **物理更新**：更新玩家（含加速度、摩擦、重力）、子弹（直线运动）、炸弹（抛物线 + 倒计时）的位置和碰撞状态。
3. **规则判定**：依次进行子弹命中检测、炸弹爆炸检测、生命状态检测。
4. **效果更新**：粒子特效帧计数衰减。
5. **画面渲染** (`paintEvent`)：绘制渐变背景、平台（草地纹理）、角色（精灵图 + 方向翻转 + bob 动画 + 无敌闪烁）、弹药、特效粒子和 UI 信息（血量、弹夹、击杀提示、屏幕外标记）。

AI 输入方面，每 `AI_DECISION_INTERVAL=5` 帧收集一次 21 维游戏状态向量，交由 `AIController` 决策后映射为 8 种离散动作之一 (`Idle/MoveLeft/MoveRight/Jump/Shoot/Bomb/MoveLeft`).

2.5 AI 系统

2.5.1 网络结构 (SimpleMLP)

C++ 端手写实现多层感知机前向传播，网络结构为 $21 \rightarrow 128 \rightarrow 128 \rightarrow 8$ ，隐藏层使用 ReLU 激活，输出层无激活（直接输出 Q 值）。权重从 `weights.txt` 文本文件加载（约 337 行），不依赖任何深度学习框架。推理时取 Q 值最大的动作 (`argmax`)。

2.5.2 AIController

提供统一决策接口 `decideAction(state)`。若模型加载成功则使用神经网络推理；否则回退到**规则 AI** (`fallbackRuleAction`)，保证游戏始终可玩。简单模式强制使用规则 AI。

规则 AI 的策略优先级：躲避来袭子弹 → 近距离投弹 → 对齐射击 → 追踪敌人 → 跳跃接近。

2.5.3 状态空间

21 维归一化浮点向量，包含：自身位置/速度/HP/着地/朝向（7 维）、敌方属性（5 维）、敌我相对距离（2 维）、射击/炸弹冷却（2 维）、最近敌方子弹和炸弹位置（5 维）。

2.6 强化学习训练系统 (Python 端)

2.6.1 算法: Double DQN

采用 **Double Deep Q-Network** 算法, 使用策略网络选择动作、目标网络评估 Q 值, 减少过乐观估计:

$$y = r + \gamma \cdot Q_{\text{target}}\left(s', \arg \max_{a'} Q_{\text{policy}}(s', a')\right)$$

损失函数为 Smooth L1 (Huber Loss), Adam 优化器 ($lr = 10^{-3}$), 梯度裁剪阈值 5.0。

2.6.2 分阶段课程学习

阶段	Episode	对手与目的
1	1-300	随机对手, 学习基础移动和攻击
2	301-900	规则 AI, 学习战术对抗
3	901-1800	混合对手 (规则 + 随机), 提升鲁棒性
4	1801-3500	Self-play (对战自身历史快照), 持续进化

ϵ -greedy 探索从 1.0 线性衰减至 0.05 (2200 episodes 内)。经验回放池容量 120,000, 批大小 128, 折扣因子 $\gamma = 0.98$, 目标网络每 25 个 episode 同步一次。

2.6.3 奖励设计

奖励函数综合多个维度: 每步微小惩罚 (-0.01) 鼓励快速行动; 伤敌奖励 ($+1.0$) 与被伤惩罚 (-1.25) 的非对称设计鼓励攻击但惩罚冒进; 胜负奖励 (± 150.0) 提供强烈终局信号。此外还包括距离 ($+0.05$)、射击对齐 ($+0.04$)、近距离投弹 ($+0.04$)、无效投弹 (-0.06) 等塑形奖励。

2.6.4 环境与部署

Python 端 (`duel_env.py`) 完整复刻了 C++ 的物理和规则系统。训练完成后通过 `export_weights.py` 将 PyTorch 模型导出为文本格式 `weights.txt`, 供 C++ 端 `SimpleMLP::loadWeights` 直接加载。

2.7 音频与 UI 系统

AudioManager 采用单例模式, 使用 `QSoundEffect` 播放 WAV 音效 (射击、爆炸), `QMediaPlayer` 播放背景音乐 (M4A), 支持音效/音乐独立音量调节和启用/禁用切换。

MainWindow 使用 `QStackedWidget` 实现菜单与游戏界面的切换, 提供渐变色主菜单。**PauseDialog** 支持暂停时调整音量和游戏控制。**GameConfig** 以单例模式管理全局按键映射 (P1: WASD+TY, P2: 方向键 + 逗号句号)。

3 小组成员分工

本项目采用**按功能模块分工**的方式, 每位成员负责完整的功能边界, 避免按文件类型切分导致的职责不清。核心原则: A 不写游戏规则只管玩家和攻击, B 不管胜负判断只管显示和输入, C 不管界面美化只管碰撞和规则。

3.1 李子禾 — 玩家与攻击系统负责人

负责文件: `gameobject.h/cpp`、`player.h/cpp`、`bullet.h/cpp`、`bomb.h/cpp`

主要工作:

- 设计并实现 `GameObject` 基类, 提供浮点坐标系统和 `QRect/QRectF` 双精度碰撞矩形。
- 实现 `Player` 类的完整物理系统: 加速度模型 (地面/空中不同参数)、摩擦力、重力、二段跳机制、平台穿越。
- 实现武器系统: 弹夹管理 (8 发/78 帧装弹)、射击冷却 (9 帧)、炸弹冷却 (132 帧)、后坐力和击飞效果。
- 实现 `Bullet` 类: 直线飞行、伤害属性、击退参数、生命周期管理。
- 实现 `Bomb` 类: 抛物线运动物理 (重力 0.42)、倒计时 (105 帧)、爆炸动画 (28 帧)、范围伤害与基于角度的方向性击飞计算。
- 实现玩家状态管理: HP 扣减、无敌计时、死亡判定、受击闪烁动画。

3.2 吴狄隆 — 界面与游戏画布负责人

负责文件: `mainwindow.h/cpp`、`gamewidget.h/cpp` (绘制与输入部分)、`resources.qrc`、`audiomanager.h/cpp`

主要工作:

- 搭建 Qt 项目框架, 创建 `MainWindow` 主窗口和 `GameWidget` 游戏画布。
- 实现 `QTimer` 驱动的 60FPS 游戏循环和 `paintEvent` 渲染管线。
- 实现双人键盘输入处理系统 (`KeyPressEvent/keyReleaseEvent`, `QSet` 按键状态管理)。
- 完成所有 UI 界面: 渐变色主菜单、游戏模式选择对话框、暂停菜单 (音量控制)、帮助说明界面、游戏结束/胜负显示。
- 实现视觉效果系统: 着陆尘埃、枪口火焰、命中闪光、爆炸粒子、浮动伤害文字、屏幕震动、击杀提示、屏幕外敌人标记。
- 集成音频系统 (`AudioManager`), 管理背景音乐和各类音效的播放。
- 负责精灵图、背景、平台纹理等美术资源的制作与集成。

3.3 边扬博 — 地图、碰撞、规则与 AI 负责人

负责文件: `platform.h/cpp`、`gamemanager.h/cpp`、`ai_controller.h/cpp`、`simple_mlp.h/cpp`、`ai_types.h`、`gameconfig.h`, 以及全部 Python 训练代码

主要工作:

- 实现 `Platform` 类和 9 平台对称竞技场地图设计。
- 实现 `GameManager` 中的碰撞检测系统: 玩家-平台碰撞 (含下穿机制)、子弹-平台碰撞 (火花特效)、炸弹-平台碰撞。
- 实现规则判定系统: 子弹命中检测 (含无敌免疫)、炸弹范围伤害检测 (圆形碰撞 + 角度击飞)、生命状态管理 (坠亡/血量归零/复活/游戏结束判定)。
- 设计并实现 AI 系统: 21 维状态空间设计、8 种离散动作定义、规则 AI (fallback 策略)、C++ 端 `SimpleMLP` 前向传播推理 (手写矩阵运算, 零依赖)。
- 搭建完整的 Python 强化学习训练框架: `Double DQN` 算法实现、游戏环境模拟 (`duel_env.py` 完整复刻 C++ 物理)、分阶段课程学习策略、奖励函数设计、经验回放和 Self-play 机

制。

- 完成权重导出工具 (`export_weights.py`)，实现 Python 训练到 C++ 部署的完整 workflow。

3.4 分工汇总

成员	角色定位	最终交付
李子禾	玩家与攻击系统	玩家能移动/射击/投弹、被击退、HP 和死亡正常
吴狄隆	界面与游戏画布	游戏能显示、能操作、界面完整、音效正常
边扬博	地图、规则与 AI	碰撞和规则完整、AI 可对战、训练管线可用

4 项目总结与反思

4.1 项目成果

本项目成功实现了一个**端到端的 AI 对战游戏系统**，涵盖游戏开发与人工智能两大领域：

- 完成了功能完整的 Qt 双人对战游戏，包含精细的物理引擎、碰撞系统、武器系统、丰富的视觉效果和音频系统。
- 设计并实现了基于 Double DQN 的强化学习训练框架，通过分阶段课程学习和自我对弈训练出了具有实战能力的 AI 对手。
- 实现了从 Python 训练到 C++ 部署的完整 workflow：AI 模型以文本权重文件嵌入游戏，不依赖任何深度学习框架，部署代码仅约 120 行。
- 三人按功能模块清晰分工，通过预先约定的类接口协作开发，代码结构整洁、模块边界清晰。

4.2 技术亮点

1. **训练-部署一致性**：Python 训练环境严格对齐 C++ 物理参数（速度、加速度、重力、伤害等全部一致），确保训练行为与实际游戏表现匹配。
2. **分阶段课程学习**：从随机对手逐步过渡到规则 AI 再到自我对弈，避免训练早期陷入局部最优。
3. **轻量级部署**：C++ 端手写 MLP 前向传播，通过文本权重文件加载模型，零外部依赖，可在任意平台编译运行。
4. **鲁棒性设计**：AI 系统具备规则回退机制，模型加载失败时自动切换到规则 AI，保证游戏正常运行。
5. **接口驱动协作**：三人开发前预先定义了 Player、Bullet、Bomb、Platform、GameManager 等类的公共接口，有效减少了整合冲突。

4.3 不足与反思

1. **算法局限**：DQN 属于值函数方法，离散动作空间（8 种）无法表达”边跳边射”等组合操作。未来可尝试 PPO 或 Actor-Critic 架构，并扩展动作空间以提升 AI 操作上限。

2. **状态空间有限**: 21 维状态未包含弹药数、多颗子弹/炸弹的完整信息。更丰富的状态表示或引入注意力机制可能有助于 AI 学习更精细的战术。
3. **训练效率**: 纯 CPU 上的 Python 环境模拟速度有限。可通过 Cython 加速环境模拟或使用向量化并行环境提高训练吞吐量。
4. **评估不足**: 缺少与人类玩家的系统性对比测试。后续可引入 Elo 评分等定量评估方法。
5. **扩展功能**: 分工表中规划的夺旗模式 (Flag/Base) 由于时间限制未能实现, 可作为后续扩展方向。

4.4 收获与体会

通过本项目, 小组成员在以下方面获得了实践经验:

- **Qt 游戏开发**: 事件循环与定时器驱动、QPainter 渲染管线、QSoundEffect/QMediaPlayer 音频、QRC 资源管理。
- **强化学习实践**: DQN 及其变体的原理与工程实践, 包括经验回放、目标网络、 ϵ 衰减、奖励塑形等关键技术。
- **跨语言系统集成**: Python 训练 $\rightarrow$ 权重导出 $\rightarrow$ C++ 推理的完整部署流程, 以及两端物理参数对齐的工程挑战。
- **团队协作**: 按功能模块分工、接口先行的协作方式, 以及 Git 版本控制下的多人开发实践。